

Guía de estudio

Programación Avanzada — Primer examen parcial

Prof. Gustavo Márquez Flores · Semestre 2026-1

CUADERNO DE SOLUCIONES

Cada problema aparece con su enunciado y su solución desarrollada paso a paso.

Contenido

1. Fundamentos: lenguajes, gramáticas, paradigmas, programación estructurada y modular, métricas de módulos.
2. Divide y vencerás: esquema general, búsqueda binaria, QuickSort, multiplicación de enteros grandes.
3. Backtracking: una, todas y la mejor solución; ocho reinas; cuadros mágicos y construcción por patrones.
4. Lógica de Hoare básica: aserciones, fortaleza, corrección parcial/total, asignación, consecuencia, conjunción.
5. Composición secuencial e invariantes de bloque.
6. Condicionales (if con y sin **else**).
7. Ciclos: inducción, invariante de ciclo, salida y terminación.
8. Recursión doble: función 91 de McCarthy, inducción modificada y función de Ackermann.
9. Programación lógica: cláusulas de Horn, resolución, SLD, unificación, backtracking y corte en Prolog.
10. Dos exámenes simulados (formato de 5–6 preguntas, como el del año pasado).

Los problemas marcados con ★ son *literalmente* las preguntas del examen del año pasado.

Cómo es el examen del profesor

El primer parcial del año pasado tuvo **cinco preguntas** que combinan demostración formal, teoría breve y pseudocódigo:

#	Pregunta	Tipo
1	Demostrar que $\{a < b\}$ if $(a > b)$ then $m := a$ else $m := b$ $\{m = b\}$ es válida.	Demostración (Hoare)
2	¿Qué métricas de módulos existen para medir su eficiencia?	Teoría breve
3	Escribir en pseudocódigo el algoritmo de <i>Divide y vencerás</i> .	Pseudocódigo
4	Demostrar que el invariante de <code>int j=9; for(int i=0; i<10; i++) j-;</code> es $i + j = 9$.	Demostración (inducción)
5	Describir el algoritmo de <i>Backtracking</i> para buscar la <i>mejor</i> solución.	Pseudocódigo + teoría

Qué se deduce: el profesor pide (i) demostraciones con reglas de Hoare escritas completas, (ii) invariantes de ciclos probados por inducción, (iii) los esquemas de sus hojas “*Algoritmo BackTracking y Divide-Vencerás*” en su misma notación y (iv) definiciones cortas. Como este año el curso ya avanzó a la función 91, Ackermann y Prolog, es razonable esperar al menos una pregunta de esos temas.

Lección del examen corregido

En la pregunta 1 el alumno supuso $a > b$ y trató de “forzar” $m = b$; el profesor anotó “revisar planteamiento”. La clave era **reconocer que la rama then es inalcanzable**: $a < b \wedge a > b \equiv \perp$, y una terna con precondition $\perp$ es válida por vacuidad. Siempre escribe explícitamente P , B , Q , las dos ramas y la regla que usas.

Plantillas que conviene memorizar

Plantilla: demostrar una terna con condicional

1. Identificar P (precondición), B (guarda) y Q (postcondición).
2. **Rama verdadera:** con precondition $P \wedge B$, calcular $wp(C_1, Q)$ por sustitución y probar $P \wedge B \Rightarrow wp(C_1, Q)$ (regla de consecuencia). Si $P \wedge B \equiv \perp$, la rama es inalcanzable y la terna vale por vacuidad.
3. **Rama falsa:** igual con $P \wedge \neg B$ y C_2 (si no hay **else**: probar $P \wedge \neg B \Rightarrow Q$).
4. Concluir con la regla condicional. Sin ciclos ni recursión, el código termina: corrección total.

Plantilla: demostrar un invariante de ciclo

1. Traducir el **for** a **while** y nombrar valores tras n iteraciones: x_n .
2. **Caso base** ($n = 0$): el invariante I vale antes de la primera iteración.
3. **Hipótesis de inducción:** I vale tras n iteraciones.
4. **Paso inductivo:** si la guarda B es cierta y se ejecuta el cuerpo, I vale tras $n + 1$ iteraciones

(equivalente: $\{I \wedge B\}$ cuerpo $\{I\}$).

5. **Salida:** $I \wedge \neg B \Rightarrow Q$. **Terminación:** variante entera ≥ 0 que decrece.

Formulario de repaso

Lógica de Hoare

Terna	$\{P\} C \{Q\}$: si C inicia en un estado que satisface P y <i>termina</i> , el estado final satisface Q .
Corrección	total = parcial + terminación.
Fortaleza	R es más fuerte que S si $R \Rightarrow S$. $\perp$ (ningún estado) es la más fuerte; $\top \equiv \{ \}$ (todos) la más débil.
Asignación	$\{Q[E/V]\} V := E \{Q\}$ (sustituir V por E en Q ; añadir condiciones de dominio, p. ej. divisor $\neq 0$).
Consecuencia	$\frac{P_1 \Rightarrow P \quad \{P\} C \{Q\}}{\{P_1\} C \{Q\}} \quad \frac{\{P\} C \{Q\} \quad Q \Rightarrow Q_1}{\{P\} C \{Q_1\}}$
Conjunción / disyunción	$\frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \wedge P_2\} C \{Q_1 \wedge Q_2\}} \quad \frac{\{P_1\} C \{Q_1\} \quad \{P_2\} C \{Q_2\}}{\{P_1 \vee P_2\} C \{Q_1 \vee Q_2\}}$
Composición	$\frac{\{P\} C_1 \{R\} \quad \{R\} C_2 \{Q\}}{\{P\} C_1; C_2 \{Q\}} \quad (R: \text{aserción intermedia; se calcula hacia atrás}).$
if sin else	$\frac{\{P \wedge B\} C_1 \{Q\} \quad (P \wedge \neg B) \Rightarrow Q}{\{P\} \text{ if } B \text{ then } C_1 \{Q\}}$
if con else	$\frac{\{P \wedge B\} C_1 \{Q\} \quad \{P \wedge \neg B\} C_2 \{Q\}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}$
while	$P \Rightarrow I; \{I \wedge B\} C \{I\}; I \wedge \neg B \Rightarrow Q$; variante $V \in \mathbb{N}$ que decrece estrictamente.
Invariante de bloque	$\{I\} C \{I\}$: se conserva la <i>relación</i> , no los valores.

Inducción, McCarthy y Ackermann

Inducción ordinaria: $P(k)$ y $\forall n \geq k (P(n) \Rightarrow P(n+1)) \Rightarrow \forall n \geq k P(n)$.

Inducción modificada (hacia abajo): $P(k)$ y, para todo $n < k$, $[P(m)$ para todo m con $n < m \leq k] \Rightarrow P(n) \Rightarrow \forall n \leq k P(n)$.

Función 91: $f(x) = x - 10$ si $x > 100$; $f(x) = f(f(x+11))$ si $x \leq 100$. Teorema: $f(x) = 91$ para $x \leq 100$.

Ackermann: $A(0, y) = y + 1$; $A(x+1, 0) = A(x, 1)$; $A(x+1, y+1) = A(x, A(x+1, y))$.

$A(1, z) = z + 2$, $A(2, z) = 2z + 3$, $A(3, z) = 2^{z+3} - 3$, $A(4, z) = \underbrace{2^{2^{\cdot^{\cdot^2}}}}_{z+3 \text{ doses}} - 3$ ($A(4, 0) = 13$, $A(4, 1) = 65533$).

Técnicas de diseño

Divide y vencerás: caso base simple; descomponer en k subproblemas *semejantes y menores*; resolver recursivamente; *combinar*.

Recurrencias típicas: $T(n) = T(n/2) + \Theta(1) \Rightarrow \Theta(\log n)$ (búsqueda binaria); $2T(n/2) + \Theta(1) \Rightarrow \Theta(n)$ (máximo); $2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ (mergesort, QuickSort balanceado); $T(n-1) + \Theta(n) \Rightarrow \Theta(n^2)$ (QuickSort peor caso); $4T(n/2) + \Theta(n) \Rightarrow \Theta(n^2)$; $3T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3})$ (Karatsuba).

Backtracking: árbol de decisiones recorrido en profundidad; *elegir – validar (poda) – anotar – avanzar – deshacer*. *Una:* se detiene con el primer éxito. *Todas:* registra y sigue. *Mejor:* compara con la mejor guardada y sigue (con cota $\rightarrow$ *branch and bound*).

Ocho reinas: una reina por fila; columna j , diagonales $j-k$ y $j+k$. 92 soluciones (12 fundamentales). $n=4$: 2 soluciones.

Cuadro mágico: $M_n = \frac{n(n^2+1)}{2}$ ($M_3 = 15$, $M_4 = 34$, $M_5 = 65$). Impar: método siamés (arriba-derecha; si está ocupada, abajo). $n \equiv 0 \pmod{4}$: llenar $1..n^2$ y cambiar $x \mapsto n^2 + 1 - x$ fuera de las diagonales de cada bloque 4×4 .

Prolog y resolución

Regla $a :- b_1, \dots, b_m. \equiv b_1 \wedge \dots \wedge b_m \rightarrow a \equiv a \vee \neg b_1 \vee \dots \vee \neg b_m$ (cláusula de Horn definida).

Meta $?- a_1, \dots, a_n. \equiv \leftarrow A_1, \dots, A_n \equiv \neg A_1 \vee \dots \vee \neg A_n$.

Resolución: $\frac{C \vee L \quad D \vee \neg L'}{(C \vee D)\theta}$, $\theta = \text{UMG}(L, L')$. Refutación: $S \cup \{\neg F\} \vdash \square \Rightarrow S \models F$.

Paso SLD: $\leftarrow A_1, \dots, A_k, \dots, A_n$ con regla $A \leftarrow B_1..B_m$ y $\theta = \text{UMG}(A, A_k)$ da $\leftarrow (A_1..A_{k-1}, B_1..B_m, A_{k+1}..A_n)\theta$. Éxito: meta vacía $\square$.

Sintaxis: $,$ conjunción; $;$ disyunción (u otra respuesta en consola); $\neg$ negación como falla; $!$ corte; is evalúa aritmética; $=$ solo unifica; $==$ compara valores; Mayúscula inicial o $_$ = variable; minúscula = átomo; sin espacio entre predicado y paréntesis.

Tema 1. Fundamentos y técnicas de programación

Problema 1.1

¿Qué significa que un programa sea *correcto*? Explica por qué “correcto” y “eficiente” no son sinónimos y menciona tres criterios con los que suele juzgarse un programa.

Solución

Un programa es **correcto respecto de una especificación** (precondición P y postcondición Q) si, para toda entrada que satisface P , produce una salida que satisface Q (corrección parcial) y además siempre termina (corrección total). La corrección es *relativa* a la especificación: no existe “correcto en abstracto”.

La eficiencia mide *cuántos recursos* usa el programa (tiempo, memoria). Un programa puede ser correcto y lentísimo (p. ej. ordenar probando todas las permutaciones) o rápido e incorrecto.

Criterios (preguntas del “tour” del curso): ¿está bien escrito y es legible?, ¿es óptimo en **tiempo** y **memoria**?, ¿usa eficientemente el lenguaje?, ¿el algoritmo es eficiente?, ¿se puede **demostrar** correcto?

Problema 1.2

Define *algoritmo* y *programa*. ¿Qué expresa la fórmula de Niklaus Wirth?

Solución

Algoritmo: método de solución con un número finito de pasos bien definidos; puede recibir datos, produce al menos un resultado y termina en tiempo finito. **Programa:** un algoritmo expresado en un lenguaje de programación (código fuente sintácticamente válido) para que una computadora lo ejecute.

Wirth: *Algoritmos + Estructuras de datos = Programas*. Resolver un problema no es solo describir pasos: también hay que decidir cómo representar y organizar la información sobre la que operan.

Problema 1.3

Sea $G = (V, T, S, P)$ con $T = \{a, b\}$, $N = \{S\}$ y producciones $S \rightarrow aSb \mid ab$.

- Explica qué es cada componente de G .
- Deriva la cadena $aaabbb$. ¿Cuál es $L(G)$?
- ¿De qué tipo es G en la jerarquía de Chomsky? ¿Y la gramática $S \rightarrow aS \mid b$?

Solución

(a) V : vocabulario finito; $T \subseteq V$: terminales; $N = V \setminus T$: no terminales; $S \in N$: símbolo inicial; P : reglas de producción.

(b) $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb$. $L(G) = \{a^n b^n \mid n \geq 1\}$.

(c) Cada producción tiene un único no terminal a la izquierda: **tipo 2, libre de contexto**. No es regular (tipo 3): $a^n b^n$ exige “contar”, cosa que un autómata finito no puede. La gramática $S \rightarrow aS \mid b$ tiene producciones $A \rightarrow aB$ o $A \rightarrow a$: **tipo 3, regular**; genera $L = \{a^n b \mid n \geq 0\}$.

Tabla: tipo 0 irrestricta, tipo 1 sensible al contexto, tipo 2 libre de contexto, tipo 3 regular. La sintaxis de casi todos los lenguajes de programación es libre de contexto; comprobaciones como

“variable declarada” o “tipos compatibles” requieren análisis adicional.

Problema 1.4

Clasifica por paradigma (imperativo / declarativo funcional / declarativo lógico) y por forma de ejecución típica: C, Pascal, Java, Haskell, Lisp, Prolog, Python. ¿Por qué la clasificación compilado/interpretado no es absoluta?

Solución

Lenguaje	Paradigma	Ejecución típica
C, Pascal	imperativo (estructurado)	compilado
Java	imperativo, orientado a objetos	compilado a bytecode + JVM (interpreta/JIT)
Python	imperativo / OO (multiparadigma)	interpretado
Haskell	declarativo funcional	compilado (GHC) o interpretado (GHCi)
Lisp	declarativo funcional	interpretado o compilado
Prolog	declarativo lógico	interpretado (SWI) o compilado (GNU Prolog)

Imperativo: describe *cómo* cambiar el estado. Declarativo: describe *qué* relación o resultado se quiere. La forma de ejecución depende de la *implementación*, no del lenguaje: un mismo lenguaje puede tener compilador, intérprete y JIT.

Problema 1.5

Enuncia el teorema de Böhm–Jacopini y da un ejemplo en pseudocódigo de cada estructura.

Solución

Todo algoritmo (programa con un punto de entrada y uno de salida) puede expresarse combinando solo tres estructuras de control:

1. **Secuencia:** $x := 1; y := x + 2.$
2. **Selección:** $\text{if } x > 0 \text{ then } y := x \text{ else } y := -x.$
3. **Iteración:** $\text{while } i < n \text{ do } i := i + 1.$

Es la base de la programación estructurada (sin goto). El teorema no garantiza legibilidad: los nombres, la modularidad y la organización siguen importando.

Problema 1.6 (★ pregunta del examen del año pasado)

¿Qué métricas de módulos existen para medir su eficiencia (y calidad)?

Solución

Métricas de diseño modular vistas en clase:

- **Cohesión:** medida de qué tan relacionadas están las responsabilidades *internas* de un módulo. Se busca **alta** cohesión (el módulo hace una sola cosa bien definida), lo que facilita nombrarlo, entenderlo, probarlo y reutilizarlo.
- **Acoplamiento:** medida de la *dependencia* de un módulo respecto de otros. Se busca **bajo**

acoplamiento, para poder cambiar un módulo sin afectar a los demás.

Objetivo de diseño: **alta cohesión y bajo acoplamiento**.

Métricas de eficiencia del módulo:

- **Tiempo de ejecución** / complejidad temporal (p. ej. $O(n \log n)$).
- **Uso de memoria** / complejidad espacial.

Métrica de corrección: si el módulo es correcto (corrección parcial) y si termina (corrección total).

Complemento (útil si piden ejemplos, no desarrollado en clase): tipos de cohesión de Yourdon–Constantine (funcional, secuencial, comunicacional, procedimental, temporal, lógica, coincidental; de mejor a peor) y de acoplamiento (de datos, por estampado, de control, externo, común, de contenido; de mejor a peor); complejidad ciclomática de McCabe $V(G) = E - N + 2$; líneas de código; *fan-in/fan-out*.

Problema 1.7

Las diapositivas advierten que “alta cohesión $\Rightarrow$ alto acoplamiento” no es la meta. Explica la idea y describe cómo MVC reparte responsabilidades entre módulos.

Solución

Dividir un sistema en módulos muy cohesivos obliga a que se comuniquen, lo que *podría* aumentar el acoplamiento. La meta es lograr alta cohesión *sin* dependencias innecesarias: módulos que hablan mediante interfaces pequeñas y bien definidas.

MVC: *Controlador* recibe la solicitud y coordina (p. ej. servlet); *Modelo* contiene datos y lógica del dominio (JavaBeans/EJB, acceso a datos); *Vista* presenta el resultado (JSP). Cada parte tiene una responsabilidad (alta cohesión) y la vista puede cambiar sin tocar la lógica del dominio (bajo acoplamiento).

Tema 2. Divide y vencerás

Problema 2.1 (★ pregunta del examen del año pasado)

Escribe en pseudocódigo el algoritmo de *Divide y vencerás* para resolver un problema y explica las condiciones que debe cumplir el problema para aplicarlo.

Solución

Esquema del profesor:

```

Funcion Divide_y_Vencerás( P : Problema ): Solución
{ Devuelve la solución del problema P }
Inicio
  Si P es suficientemente pequeño o simple
    Entonces devolver( solucion_simple( P ) )
  Si_No
    descomponer P en k partes( P1, P2, ..., Pk );
    Para( i = 1; i <= k; i++ )
      Si = Divide_y_Vencerás( Pi );

```

```

    devolver( combinar( S1, S2, ..., Sk ) );
Fin

```

Condiciones:

1. P se puede descomponer en subproblemas $P_1, \dots, P_k$ **semejantes** al original y **de menor tamaño**.
2. Existe un **caso base** que se resuelve directamente (`solucion_simple`).
3. Las subsoluciones se pueden **combinar** eficientemente.
4. Cada llamada reduce una medida (p.ej. el tamaño), lo que garantiza la **terminación**.

Costo: $T(n) = \sum_i T(n_i) + D(n) + C(n)$ (dividir + combinar); con a subproblemas de tamaño n/b : $T(n) = aT(n/b) + f(n)$.

Ejemplos: QuickSort, búsqueda binaria, mergesort, multiplicación de enteros grandes.

Ojo: en la hoja del “tour” aparece `combinar(P1, ..., Pk)`; lo correcto es combinar las *soluciones* $S_1, \dots, S_k$.

Problema 2.2

Escribe la búsqueda binaria recursiva sobre un arreglo ordenado $A[\text{izq}..\text{der}]$. Traza la búsqueda de $X = 23$ y de $X = 10$ en $A = [3, 8, 12, 17, 23, 31, 40, 52]$ (índices 1..8). ¿Por qué las llamadas usan $m - 1$ y $m + 1$? ¿Cuál es su complejidad?

Solución

```

Funcion BusBin( A, X, izq, der ): entero
Inicio
    Si izq > der entonces devolver( 0 )           // caso vacío: no está
    m = (izq + der) div 2
    Si X = A[m] entonces devolver( m )           // caso base
    Si X < A[m] entonces devolver( BusBin(A, X, izq, m-1) )
    Si_No devolver( BusBin(A, X, m+1, der) )
Fin

```

$X = 23$

$[1, 8], m = 4, A[4] = 17 < 23 \rightarrow [5, 8]$

$[5, 8], m = 6, A[6] = 31 > 23 \rightarrow [5, 5]$

$[5, 5], m = 5, A[5] = 23 \Rightarrow$ devuelve 5

$X = 10$

$[1, 8], m = 4, A[4] = 17 > 10 \rightarrow [1, 3]$

$[1, 3], m = 2, A[2] = 8 < 10 \rightarrow [3, 3]$

$[3, 3], m = 3, A[3] = 12 > 10 \rightarrow [3, 2]:$ vacío $\Rightarrow 0$

Usar $m \pm 1$ excluye el elemento ya comparado; así el intervalo *se reduce estrictamente* en cada llamada y la recursión termina (con m en lugar de $m - 1$ podría repetirse el mismo intervalo indefinidamente). Es divide y vencerás con un solo subproblema y combinación trivial: $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$. Requiere arreglo **ordenado**.

Problema 2.3

Diseña con divide y vencerás una función `Maximo(A, l, r)` que devuelva el mayor elemento de $A[l..r]$. Plantea la recurrencia y resuélvela.

Solución

```

Funcion Maximo( A, l, r ): valor
Inicio
  Si l = r entonces devolver( A[l] )           // caso simple
  m = (l + r) div 2                             // descomponer en 2
  M1 = Maximo( A, l, m )
  M2 = Maximo( A, m+1, r )
  Si M1 >= M2 entonces devolver( M1 )         // combinar
  Si_No devolver( M2 )
Fin

```

$T(1) = \Theta(1)$, $T(n) = 2T(n/2) + \Theta(1)$. Al desarrollar: $T(n) = 2^k T(n/2^k) + (2^k - 1)c$; con $2^k = n$ queda $T(n) = \Theta(n)$ (exactamente $n - 1$ comparaciones). Terminación: $l \leq m < r$, así que ambos subintervalos son estrictamente menores.

Problema 2.4

Considera el procedimiento `Sort(l, r)` de `Qsort.PAS`:

```

i := l; j := r; x := a[(l+r) div 2];
repeat
  while a[i] < x do i := i + 1;
  while x < a[j] do j := j - 1;
  if i <= j then begin
    y := a[i]; a[i] := a[j]; a[j] := y; i := i + 1; j := j - 1;
  end;
until i > j;
if l < j then Sort(l, j);
if i < r then Sort(i, r);

```

Traza la primera partición para $a = [5, 3, 8, 1, 9, 2, 7]$ (índices 1..7) y di qué llamadas recursivas se generan. ¿Qué enseña este ejemplo sobre el pivote? Da la complejidad en el mejor y peor caso.

Solución

Pivote: $x = a[(1 + 7) \text{ div } 2] = a[4] = 1$.

- $i: a[1] = 5 \not< 1 \Rightarrow i = 1$. $j: 1 < 7, 1 < 2, 1 < 9$ retrocede; $1 \not< a[4] = 1 \Rightarrow j = 4$.
- $i \leq j$: intercambiar $a[1] \leftrightarrow a[4]$: $[1, 3, 8, 5, 9, 2, 7]$; $i = 2, j = 3$.
- $i: a[2] = 3 \not< 1 \Rightarrow i = 2$. $j: 1 < 8, 1 < 3$ retrocede; $a[1] = 1 \Rightarrow j = 1$.
- $i = 2 > j = 1$: no hay intercambio y termina el repeat.

Llamadas: $l < j$? $1 < 1$ no. $i < r$? $2 < 7$ sí $\Rightarrow \text{Sort}(2, 7)$. Continúa: $\text{Sort}(2, 7)$ con pivote 5 $\rightarrow [1, 3, 2, 5, 9, 8, 7]$, luego $\text{Sort}(2, 3)$ y $\text{Sort}(5, 7)$; resultado $[1, 2, 3, 5, 7, 8, 9]$.

Lección: tomar el elemento *central por posición* no garantiza una partición equilibrada: aquí el pivote fue el mínimo y quedaron regiones de tamaño 0 y 6. Complejidad: $T(n) = T(k) + T(n - k - 1) + \Theta(n)$. Balanceado: $\Theta(n \log n)$ (caso típico/esperado). Muy desbalanceado: $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$. Memoria de pila: $O(\log n)$ típico, $O(n)$ peor caso.

Invariante de la partición: lo recorrido a la izquierda de i es $\leq x$ y lo recorrido a la derecha de j es $\geq x$.

Problema 2.5

Multiplica $u = 1234$ y $v = 5678$ con el esquema de divide y vencerás de clase ($s = 2$, $B = 10^2$). Da la recurrencia del esquema y explica la mejora de Karatsuba.

Solución

$u = Bw + x$ con $w = 12$, $x = 34$; $v = By + z$ con $y = 56$, $z = 78$. Por distributividad

$$uv = B^2 wy + B(wz + xy) + xz.$$

$wy = 672$, $wz = 936$, $xy = 1904$, $xz = 2652$.

$$uv = 672 \cdot 10^4 + (936 + 1904) \cdot 10^2 + 2652 = 6720000 + 284000 + 2652 = \mathbf{7\,006\,652}.$$

Recurrencia: 4 multiplicaciones de la mitad de dígitos + sumas/desplazamientos lineales: $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$ — no mejora al método escolar.

Karatsuba: calcula wy , xz y $p = (w+x)(y+z)$; entonces $wz+xy = p - wy - xz$. Solo 3 multiplicaciones: $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. Aquí: $p = 46 \cdot 134 = 6164$ y $6164 - 672 - 2652 = 2840 = 936 + 1904$. ✓

Problema 2.6

Escribe *mergesort* siguiendo el esquema del profesor, señalando qué parte es “descomponer” y cuál “combinar”. Compáralo con QuickSort: ¿dónde hace cada uno el trabajo pesado?

Solución

```

Funcion MergeSort( A, l, r )
Inicio
  Si l >= r entonces devolver                // 0 o 1 elemento: ya ordenado
  m = (l + r) div 2                          // descomponer (trivial)
  MergeSort( A, l, m ); MergeSort( A, m+1, r ) // resolver
  Mezclar( A, l, m, r )                      // combinar: Theta(n)
Fin

```

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ siempre, con $\Theta(n)$ de memoria auxiliar. En **mergesort** dividir es trivial y el trabajo está en **combinar** (mezclar). En **QuickSort** el trabajo está en **dividir** (partición alrededor del pivote) y combinar es trivial (las regiones ya quedan en su lugar, *in-place*).

Problema 2.7

Responde brevemente: (a) ¿Todo algoritmo recursivo es divide y vencerás? (b) ¿Qué diferencia hay entre caso base y caso vacío? (c) ¿Cuál es la diferencia esencial entre divide y vencerás y backtracking?

Solución

(a) No. La recursión es un *mecanismo de implementación*; divide y vencerás es una *estrategia de diseño* (dividir en subproblemas semejantes, resolver, combinar). Ej.: la función 91 es recursiva pero no divide ni combina.

(b) El caso base da una solución directa (p. ej. $A[m] = X$); el caso vacío indica que ya no hay candi-

datos ($izq > der$).

(c) Divide y vencerás *necesita todas* las subsoluciones y las combina. Backtracking explora un árbol de decisiones, *abandona* ramas inviables (poda) y deshace decisiones para probar alternativas; busca una, todas o la mejor solución.

Tema 3. Backtracking, ocho reinas y cuadros mágicos

Problema 3.1 (★ pregunta del examen del año pasado)

Describe el algoritmo de *Backtracking* cuando se trata de buscar la **mejor** solución de un problema. Indica en qué difiere de las versiones que buscan *una* y *todas* las soluciones.

Solución

Backtracking construye la solución por **etapas** i ; en cada una prueba las opciones disponibles, **anota** solo las *aceptables* (poda), avanza a la etapa $i + 1$ y, al regresar, **cancela la anotación** para probar la siguiente opción (vuelta atrás). Recorre en profundidad el árbol de soluciones parciales. Versión *mejor solución* (notación de la hoja del profesor):

```

Funcion Backtracking( Etapa i, solucionMejor ) devuelve: boolean
Inicio
    Éxito = falso;
    IniciarOpciones( i, GrupoOpciones o );
    Repetir
        SeleccionarNuevaOpcion( o, Opcion n );
        Si ( Aceptable( n ) ) entonces
            AnotarOpcion( i, n );
            Si SolucionCompleta( i ) entonces
                Éxito = verdadero;
                Si EsMejorSolucion( solucionActual, solucionMejor ) entonces
                    ApuntarSolucionActual();          // solucionMejor := solucionActual
                finsi;
            Sino
                Éxito = Backtracking( i+1, solucionMejor );
                Si Éxito = falso entonces cancelamosAnotacion( i, n ); finsi;
            Finsi;
        Finsi;
    Hasta ( NoQuedanOpciones( o ) );          // NO se detiene al primer éxito
    Retorna Éxito;
Fin

```

Diferencias:

- **Una solución:** el ciclo termina con Hasta (Éxito = verdadero) o NoQuedanOpciones(o): se detiene en la primera hoja válida.
- **Todas:** el ciclo es Hasta NoQuedanOpciones(o); cada solución completa se guarda con ApuntarSoluciónCompleta(i, solucionesCompletas) y se sigue explorando.
- **Mejor:** también explora todo el árbol, pero en vez de guardar todas compara cada solución completa con la mejor encontrada (EsMejorSolucion) y conserva solo la mejor. Requiere un

criterio de calidad. Si además se podan ramas cuya cota no puede superar a la mejor actual, se obtiene *ramificación y acotamiento* (branch and bound).

Detalle fino: para seguir explorando después de registrar una solución hay que deshacer la anotación *siempre* (no solo cuando falla la llamada); en muchas implementaciones se escribe `cancelamosAnotacion(i, n)` al final de cada intento. Costo sin poda efectiva: $O(b^d)$ nodos (ramificación b , profundidad d).

Problema 3.2

Problema de las 4 reinas (una reina por fila, filas $k = 0..3$, columnas $j = 1..4$).

- ¿Por qué colocar una reina por fila elimina la restricción de filas sin comprobarla?
- ¿Qué identifican $j - k$ y $j + k$? ¿Se atacan $(k, j) = (0, 2)$ y $(2, 4)$? ¿Y $(1, 4)$ y $(3, 2)$?
- Traza el árbol de búsqueda en profundidad (probando columnas de 1 a 4) hasta la primera solución.
- ¿Cuántas soluciones tiene el problema de 4 reinas? ¿Y el de 8? ¿Qué son las 12 soluciones fundamentales?

Solución

(a) Cada nivel k de la recursión decide solo la columna de la fila k ; es imposible poner dos reinas en la misma fila, así que esa restricción se cumple *por construcción*. La solución se representa como $sol[k] = j$.

(b) $j - k$ es constante en cada diagonal “\” (45° en el código) y $j + k$ en cada diagonal “/” (135°). $(0, 2)$ y $(2, 4)$: $2 - 0 = 2 = 4 - 2 \Rightarrow$ **se atacan**. $(1, 4)$ y $(3, 2)$: $4 + 1 = 5 = 2 + 3 \Rightarrow$ **se atacan**.

(c) Notación: $F_k = j$.

- $F_0 = 1$. F_1 : 1 (columna), 2 (diag. $j - k$: $1 = 1$) $\times$; $F_1 = 3$ ✓. F_2 : 1 col. $\times$, 2 ($j + k = 4$ vs. $(1, 3)$: 4) $\times$, 3 col. $\times$, 4 ($j - k = 2$ vs. $(1, 3)$: 2) $\times \Rightarrow$ *retroceder*.
- $F_1 = 4$ ✓. $F_2 = 2$ ✓. F_3 : 1, 2, 4 columnas ocupadas, 3 ($j - k = 0$ vs. $(2, 2)$) $\times \Rightarrow$ *retroceder*. $F_2 = 3$ ($j - k = 1$ vs. $(0, 1)$) $\times$; 4 col. $\times \Rightarrow$ *retroceder hasta F_0* .
- $F_0 = 2$. F_1 : 1 ($j + k = 2$) $\times$, 2 col. $\times$, 3 ($j - k = 2$) $\times$, $F_1 = 4$ ✓. $F_2 = 1$ ✓. F_3 : 1 col., 2 col., $F_3 = 3$ ✓ ($j - k = 0$, $j + k = 6$ libres).

Primera solución: $sol = [2, 4, 1, 3]$.

(d) 4 reinas: 2 soluciones, $[2, 4, 1, 3]$ y $[3, 1, 4, 2]$ (una es reflejo de la otra). 8 reinas: **92**; agrupando las que se obtienen por rotaciones y reflexiones del tablero quedan **12 fundamentales**. El programa de clase imprime 92 porque no elimina simetrías.

Problema 3.3

En el programa `Ocho_reinas.c` visto en clase:

```
for (int j = 1; j <= NREINAS; j++)
    if (!contiene(col, j) && !contiene(diag45, j - k) && !contiene(diag135, j + k)) {
        sol[k] = j;
        col.push_back(j); diag45.push_back(j - k); diag135.push_back(j + k);
        reinas(k + 1, col, diag45, diag135);
    }
```

```
col.pop_back(); diag45.pop_back(); diag135.pop_back();
}
```

(a) Relaciona cada línea con las operaciones del esquema de backtracking del profesor. (b) ¿Qué representa $k == NREINAS$? (c) ¿Por qué no hace falta “borrar” $sol[k]$?

Solución

(a) for $j = \text{IniciarOpciones/SeleccionarNuevaOpcion}$; el if con $\text{contiene} = \text{Aceptable}(n)$ (poda); $sol[k]=j$ y los $\text{push_back} = \text{AnotarOpcion}$; $\text{reinas}(k+1, \dots) = \text{Backtracking}(i+1)$; los $\text{pop_back} = \text{cancelamosAnotacion}$ (deshacer). El ciclo recorre todas las columnas: versión *todas las soluciones*.

(b) Que ya se colocaron las 8 reinas (filas 0..7): *SolucionCompleta*; se imprime y se regresa.

(c) $sol[k]$ se sobrescribe antes de volver a imprimir una solución completa. (Los vectores se pasan por valor, así que los pop_back expresan el retroceso aunque no son indispensables para el nivel llamador.)

Problema 3.4 (estilo examen)

Mochila 0/1: objetos con pesos $w = (2, 3, 4, 5)$ y valores $v = (3, 4, 5, 7)$, capacidad 7. Se quiere el subconjunto de mayor valor.

- Formúlalo como backtracking de *mejor solución*: ¿qué es una etapa, una opción, Aceptable, *SolucionCompleta* y *EsMejorSolucion*?
- Encuentra la mejor solución y muestra al menos una rama podada.

Solución

(a) Etapa i : decidir sobre el objeto i ($i = 1..4$). Opciones: {incluir, no incluir}. Aceptable: el peso acumulado no excede 7 (poda por factibilidad). *SolucionCompleta*: $i = 4$ (se decidió sobre todos). *EsMejorSolucion*: valor actual > mejor valor guardado. Poda adicional opcional (cota): si valor actual + suma de valores restantes $\leq$ mejor, no vale la pena seguir.

(b) Soluciones factibles relevantes: {1, 2}: peso 5, valor 7; {1, 3}: 6, 8; {1, 4}: 7, **10**; {2, 3}: 7, 9; {4}: 5, 7. Mejor: {1, 4} (pesos 2 + 5) con valor **10**.

Ramas podadas por factibilidad: tras incluir 1 y 2 (peso 5), incluir el 3 daría peso 9 > 7 $\Rightarrow$ no Aceptable; lo mismo con el 4 (peso 10). Tras {1, 3} (peso 6), incluir el 4 da 11: podado.

Poda por cota (branch and bound): cuando ya se tiene la mejor = 10, la rama “no incluir 1, no incluir 2, no incluir 3” puede lograr a lo más $v_4 = 7 < 10$, así que se descarta sin explorarla.

Problema 3.5

Cuadros mágicos.

- Deduces la constante mágica M_n y calcula M_3, M_4, M_5, M_6 .
- Construye el cuadro 3×3 y el 5×5 con el método siamés.
- Construye el 4×4 con el método de complementos.
- ¿Por qué el método siamés no sirve para $n = 4$ ni el de complementos para $n = 6$?

Solución

(a) La suma de todas las casillas es $1 + \dots + n^2 = \frac{n^2(n^2+1)}{2}$; hay n filas con la misma suma M_n : $nM_n = \frac{n^2(n^2+1)}{2} \Rightarrow M_n = \frac{n(n^2+1)}{2}$. $M_3 = 15$, $M_4 = 34$, $M_5 = 65$, $M_6 = 111$.

(b) Siamés: 1 en el centro de la fila superior; el siguiente va arriba-derecha (con “envoltura” $(f-1) \bmod n$, $(c+1) \bmod n$); si esa casilla está ocupada, se coloca *debajo* de la actual.

8	1	6
3	5	7
4	9	2

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

Todas las filas, columnas y diagonales suman 15 y 65, respectivamente.

(c) Llenar 1..16 por filas; conservar las casillas de las diagonales ($i \bmod 4 = j \bmod 4$ o $i \bmod 4 + j \bmod 4 = 3$) y sustituir las demás por $17 - x$:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

→

1	15	14	4
12	6	7	9
8	10	11	5
13	3	2	16

(todas suman 34)

(d) El siamés depende de que n sea impar (hay columna central y el recorrido diagonal cubre todas las casillas sin conflicto); el de complementos necesita bloques 4×4 completos ($n \equiv 0 \pmod{4}$). $n = 6$ es “simplemente par” y requiere otra construcción (p. ej. LUX/Strachey).

Problema 3.6

Formula la búsqueda de *todos* los cuadros mágicos 3×3 como backtracking: estado, opciones, condición de aceptación (poda) y solución completa. ¿Por qué para $n \geq 5$ es preferible la construcción por patrones si solo se quiere uno?

Solución

Estado: matriz parcialmente llena y conjunto de números usados. **Etapas:** siguiente casilla vacía (por filas). **Opciones:** números de 1..9 no usados. **Aceptable (poda):** ninguna suma parcial de fila/columna/diagonal supera 15, y toda línea que se complete suma exactamente 15. **Solución completa:** las 9 casillas llenas (y todas las líneas suman 15). Versión *todas*: registrar y seguir. Para 3×3 hay 8 soluciones (una fundamental con sus rotaciones y reflexiones).

El espacio crece como $(n^2)!$: 880 cuadros de orden 4 y 275 305 224 de orden 5. Un patrón (siamés, complementos) produce *uno* directamente en $\Theta(n^2)$, sin explorar árbol alguno. Enumerar y construir son problemas distintos.

Tema 4. Lógica de Hoare: aserciones, asignación y consecuencia

Problema 4.1

Define: aserción, estado, $s \models P$, terna de Hoare, corrección parcial y corrección total. ¿Por qué en un fragmento sin ciclos ni recursión basta la corrección parcial?

Solución

Aserción: fórmula lógica sobre las variables del programa (p. ej. $x = 3 \wedge y \leq z$). **Estado:** asignación de valores a las variables. $s \models P$: el estado s satisface P . **Terna** $\{P\} C \{Q\}$: si C empieza en un estado que satisface P y termina, el estado final satisface Q . **Parcial:** la terna es válida (“si termina, el resultado es correcto”). **Total:** parcial + se demuestra que C termina para toda entrada que cumple P .

Un fragmento sin ciclos ni recursión ejecuta un número finito de instrucciones primitivas que terminan; la terminación es inmediata y parcial $\Rightarrow$ total.

Problema 4.2

Ordena de la más fuerte a la más débil: $x \geq 0$, $\top$, $x > 10$, $\perp$, $x = 15$, $x > 0$. ¿Qué significa la aserción vacía $\{\}$? ¿Es cierto que $y \neq 0 \Rightarrow y = 4$?

Solución

$\perp \Rightarrow x = 15 \Rightarrow x > 10 \Rightarrow x > 0 \Rightarrow x \geq 0 \Rightarrow \top$. Más fuerte = admite menos estados. $\perp$ no admite ninguno (la más fuerte); $\{\} \equiv \top$ admite todos (la más débil): $\top \wedge P \equiv P$, $\top \vee P \equiv \top$. La implicación no es simétrica: $y = 4 \Rightarrow y \neq 0$, pero $y \neq 0 \not\Rightarrow y = 4$ (contraejemplo $y = 7$).

Problema 4.3

Calcula la precondition más débil P en cada caso:

- | | |
|--------------------------------------|---------------------------------------|
| (a) $\{P\} x := x + 5 \{x > 12\}$ | (b) $\{P\} y := 3x - 1 \{y \leq 8\}$ |
| (c) $\{P\} x := x \cdot x \{x < 9\}$ | (d) $\{P\} z := z/2 \{z = 5\}$ |
| (e) $\{P\} a := b \{a = b\}$ | (f) $\{P\} x := 10/(y - 2) \{x > 0\}$ |

Solución

Axioma: $P = Q[E/V]$ (sustituir la variable asignada por la expresión), más condiciones de dominio.

- (a) $x + 5 > 12 \iff x > 7$.
- (b) $3x - 1 \leq 8 \iff x \leq 3$.
- (c) $x^2 < 9 \iff -3 < x < 3$.
- (d) $z/2 = 5 \iff z = 10$.
- (e) $b = b \iff \top$: no hace falta ninguna condición; $\{ \} a := b \{a = b\}$.
- (f) $\frac{10}{y-2} > 0$ y dominio $y \neq 2$: el cociente es positivo si y solo si $y - 2 > 0$: $y > 2$.

Problema 4.4

¿Son válidas? Justifica con el axioma de asignación y la regla de consecuencia, o da un contraejemplo.

- (a) $\{x = 5\} x := x + 1 \{x > 5\}$ (b) $\{x > 0\} x := x - 1 \{x > 0\}$
 (c) $\{\top\} y := x \cdot x \{y \geq 0\}$ (d) $\{x \geq 0\} x := x - 1 \{x \geq -1\}$

Solución

- (a) $wp = x + 1 > 5 \iff x > 4$. Como $x = 5 \Rightarrow x > 4$, por **fortalecimiento de la precondition** es **válida**.
 (b) $wp = x - 1 > 0 \iff x > 1$. Pero $x > 0 \not\Rightarrow x > 1$. Contraejemplo: $x = 1$ termina con $x = 0$. **No válida**.
 (c) $wp = x^2 \geq 0 \iff \top$. **Válida** para cualquier estado.
 (d) $wp = x - 1 \geq -1 \iff x \geq 0$: la precondition dada es exactamente la wp . **Válida**.

Problema 4.5

(a) A partir de $\{y \neq 0\} x := 1/y \{x = 1/y\}$ deduce $\{y = 4\} x := 1/y \{x = 1/y\}$ escribiendo la regla usada.
 (b) A partir de $\{\top\} max := b \{max = b\}$ deduce $\{\top\} max := b \{max \geq b\}$. (c) Explica con conjuntos de estados por qué se puede *fortalecer* la precondition y *debilitar* la postcondition, pero no al revés.

Solución

- (a) Fortalecimiento de la precondition:
$$\frac{y = 4 \Rightarrow y \neq 0 \quad \{y \neq 0\} x := 1/y \{x = 1/y\}}{\{y = 4\} x := 1/y \{x = 1/y\}}.$$

 (b) Debilitamiento de la postcondition:
$$\frac{\{\top\} max := b \{max = b\} \quad max = b \Rightarrow max \geq b}{\{\top\} max := b \{max \geq b\}}.$$

 (c) Si el código funciona para *todos* los estados de P , funciona para cualquier subconjunto $P_1 \subseteq P$ (más fuerte). Si el resultado cae en el conjunto Q , también cae en cualquier superconjunto $Q_1 \supseteq Q$ (más débil). Al revés no: ampliar la precondition mete estados no verificados (en (a), $y = 0$), y restringir la postcondition puede excluir resultados reales.

Problema 4.6

Usa la regla de conjunción (y consecuencia) para demostrar $\{i = i_0 \wedge i_0 \geq 3\} i := i + 2 \{i \geq 5\}$, partiendo de las ternas $\{i = i_0\} i := i + 2 \{i = i_0 + 2\}$ y $\{i_0 \geq 3\} i := i + 2 \{i_0 \geq 3\}$. ¿Por qué la segunda terna es válida?

Solución

La segunda es válida por el axioma de asignación: i_0 es una variable lógica (el valor inicial) que no aparece asignada, así que $(i_0 \geq 3)[i + 2/i] = i_0 \geq 3$. Conjunción:
$$\frac{\{i = i_0\} i := i + 2 \{i = i_0 + 2\} \quad \{i_0 \geq 3\} i := i + 2 \{i_0 \geq 3\}}{\{i = i_0 \wedge i_0 \geq 3\} i := i + 2 \{i = i_0 + 2 \wedge i_0 \geq 3\}}.$$
 Como $i = i_0 + 2 \wedge i_0 \geq 3 \Rightarrow i \geq 5$, por debilitamiento de la postcondition: $\{i = i_0 \wedge i_0 \geq 3\} i := i + 2 \{i \geq 5\}$. ✓
 (Directo con el axioma: $wp = i + 2 \geq 5 \iff i \geq 3$.)

Tema 5. Composición secuencial e invariantes de bloque

Problema 5.1

Demuestra $\{ \} c := a + b; c := c/2 \{ c = (a + b)/2 \}$ (ejercicio del “tour” del curso).

Solución

Hacia atrás desde $Q : c = \frac{a+b}{2}$.

1. Por $c := c/2$: $Q[c/2/c] : \frac{c}{2} = \frac{a+b}{2} \iff c = a + b$. (1) $\{ c = a + b \} c := c/2 \{ c = \frac{a+b}{2} \}$.
2. Por $c := a + b$: $(c = a + b)[a + b/c] : a + b = a + b \iff \top$. (2) $\{ \top \} c := a + b \{ c = a + b \}$.
3. Composición de (2) y (1) con aserción intermedia $R : c = a + b$:

$$\frac{\{ \top \} c := a + b \{ c = a + b \} \quad \{ c = a + b \} c := c/2 \{ c = \frac{a+b}{2} \}}{\{ \top \} c := a + b; c := c/2 \{ c = \frac{a+b}{2} \}}$$

La precondition $\{ \} \equiv \top$: funciona para todos los valores de a y b .

Problema 5.2

Encuentra la precondition más débil: (a) $\{ P \} x := x + 1; y := 2x \{ y > 10 \}$; (b) $\{ P \} y := x - 3; z := y \cdot y \{ z > 16 \}$.

Solución

- (a) Por $y := 2x$: $2x > 10 \iff x > 5$. Por $x := x + 1$: $x + 1 > 5 \iff x > 4$. $P : x > 4$.
- (b) Por $z := y \cdot y$: $y^2 > 16 \iff y > 4 \vee y < -4$. Por $y := x - 3$: $x - 3 > 4 \vee x - 3 < -4 \iff x > 7 \vee x < -1$.
 $P : x > 7 \vee x < -1$.

Problema 5.3

Demuestra que el intercambio *sin variable auxiliar* es correcto: $\{ a = A \wedge b = B \} a := a + b; b := a - b; a := a - b \{ a = B \wedge b = A \}$.

Solución

A, B son constantes lógicas (valores iniciales). Hacia atrás:

1. Por $a := a - b$: $(a - b = B) \wedge (b = A)$. (3)
 2. Por $b := a - b$: sustituir b por $a - b$ en (3): $(a - (a - b) = B) \wedge (a - b = A) \iff b = B \wedge a - b = A$. (2)
 3. Por $a := a + b$: sustituir a por $a + b$ en (2): $b = B \wedge (a + b) - b = A \iff b = B \wedge a = A$. (1)
- (1) coincide con la precondition. Componiendo las tres ternas: $\{ a = A \wedge b = B \} a := a + b \{ b = B \wedge a - b = A \}$, $\{ b = B \wedge a - b = A \} b := a - b \{ a - b = B \wedge b = A \}$, $\{ a - b = B \wedge b = A \} a := a - b \{ a = B \wedge b = A \}$.
 ✓ (Con enteros de máquina podría haber desbordamiento en $a + b$.)

Problema 5.4

Demuestra que $I : s = i^2$ es invariante del bloque $i := i + 1; s := s + 2i - 1$. Comprueba con $i = 3, s = 9$.

Solución

Hay que probar $\{s = i^2\} i := i + 1; s := s + 2i - 1 \{s = i^2\}$.

1. Por $s := s + 2i - 1$: $s + 2i - 1 = i^2 \iff s = (i - 1)^2$. $\{s = (i - 1)^2\} s := s + 2i - 1 \{s = i^2\}$.
2. Por $i := i + 1$: $s = ((i + 1) - 1)^2 = i^2$. $\{s = i^2\} i := i + 1 \{s = (i - 1)^2\}$.
3. Composición $\Rightarrow \{s = i^2\} i := i + 1; s := s + 2i - 1 \{s = i^2\}$. ✓

Con valores: $i_1 = 4$, $s_1 = 9 + 2 \cdot 4 - 1 = 16 = 4^2$. El invariante no dice que los valores no cambien, sino que se conserva la *relación*.

Problema 5.5

Demuestra $\{ \} s := 1; s := s \cdot r + 2; s := s \cdot r + 3 \{s = r^2 + 2r + 3\}$ (evaluación de Horner). Observa dónde es necesaria la regla de consecuencia.

Solución

1. Por $s := s \cdot r + 3$: $wp = (s \cdot r + 3 = r^2 + 2r + 3) \iff s \cdot r = r(r + 2)$. Esta wp es incómoda (si $r = 0$ cualquier s sirve). Usamos una aserción más fuerte: $R_2 : s = r + 2$, que *implica* la wp . Por consecuencia: $\{s = r + 2\} s := s \cdot r + 3 \{s = r^2 + 2r + 3\}$.
2. Por $s := s \cdot r + 2$ con postcondición R_2 : $wp = (s \cdot r + 2 = r + 2) \iff s \cdot r = r$. Tomamos $R_1 : s = 1$, que la implica: $\{s = 1\} s := s \cdot r + 2 \{s = r + 2\}$.
3. Por $s := 1$: $1 = 1 \iff \top : \{ \top \} s := 1 \{s = 1\}$.
4. Composición de las tres. ✓

Lección: la aserción intermedia no tiene que ser la wp exacta; basta una condición más fuerte que la implique (regla de consecuencia).

Tema 6. Condicionales**Problema 6.1** (★ pregunta del examen del año pasado)

Demostrar que la siguiente terna de Hoare es válida:

$$\{a < b\} \text{ if } (a > b) \text{ then } m := a \text{ else } m := b \{m = b\}.$$

Solución

Identificamos $P : a < b$, $B : a > b$, $Q : m = b$. Por la regla del **if** con **else** basta probar (1) $\{P \wedge B\} m := a \{Q\}$ y (2) $\{P \wedge \neg B\} m := b \{Q\}$.

(1) Rama verdadera. $P \wedge B \equiv a < b \wedge a > b \equiv \perp$ (no existe estado con $a < b$ y $a > b$ a la vez). Formalmente: por asignación, $\{a = b\} m := a \{m = b\}$ (sustituir m por a en $m = b$). Como $\perp \Rightarrow a = b$ (de lo falso se sigue cualquier cosa), por fortalecimiento de la precondition:

$$\{a < b \wedge a > b\} m := a \{m = b\} \text{ es válida (por vacuidad): la rama es inalcanzable.}$$

(2) Rama falsa. $P \wedge \neg B \equiv a < b \wedge a \leq b \equiv a < b$. Por asignación: $(m = b)[b/m] \equiv b = b \equiv \top$, es decir,

$\{\top\} m := b \{m = b\}$. Como $a < b \Rightarrow \top$, por consecuencia:

$$\{a < b \wedge \neg(a > b)\} m := b \{m = b\}.$$

Conclusión. Por la regla condicional,

$$\frac{\{a < b \wedge a > b\} m := a \{m = b\} \quad \{a < b \wedge \neg(a > b)\} m := b \{m = b\}}{\{a < b\} \text{ if } (a > b) \text{ then } m := a \text{ else } m := b \{m = b\}}$$

Como no hay ciclos ni recursión, el programa termina: la terna es **totalmente correcta**. ■

Intuición: si $a < b$, la guarda $a > b$ es falsa, siempre se ejecuta $m := b$ y por tanto $m = b$.

Problema 6.2

Demuestra $\{ \} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{m \geq a \wedge m \geq b\}$.

Solución

$P = \top$, $B : a > b$, $Q : m \geq a \wedge m \geq b$.

Rama verdadera ($a > b$): $Q[a/m] = a \geq a \wedge a \geq b \equiv a \geq b$. Como $a > b \Rightarrow a \geq b$: $\{a > b\} m := a \{Q\}$.

Rama falsa ($\neg(a > b) \equiv a \leq b$): $Q[b/m] = b \geq a \wedge b \geq b \equiv b \geq a$. Como $a \leq b \Rightarrow b \geq a$: $\{\neg(a > b)\} m := b \{Q\}$.

Por la regla condicional, la terna es válida (y total). Caso límite $a = b$: se ejecuta $m := b$ y $m = a = b$ cumple Q .

Problema 6.3 (estilo examen)

Demuestra $\{a > b\} \text{ if } (a < b) \text{ then } m := b \text{ else } m := a \{m = a\}$.

Solución

$P : a > b$, $B : a < b$, $Q : m = a$. **Rama verdadera:** $P \wedge B \equiv a > b \wedge a < b \equiv \perp$: inalcanzable; $\{\perp\} m := b \{m = a\}$ es válida por vacuidad (formalmente: $wp = (b = a)$ y $\perp \Rightarrow b = a$). **Rama falsa:** $P \wedge \neg B \equiv a > b \wedge a \geq b \equiv a > b$; $wp(m := a, m = a) = (a = a) \equiv \top$ y $a > b \Rightarrow \top$. Por la regla condicional la terna es válida. ■

Problema 6.4

¿Es válida $\{ \} \text{ if } a > b \text{ then } m := a \text{ else } m := b \{m > b\}$? Si no, da un contraejemplo y propón dos formas de corregirla (cambiando solo la pre o solo la postcondición).

Solución

No es válida. Rama falsa: $\neg(a > b)$ y $m := b$ exige $(m > b)[b/m] \equiv b > b \equiv \perp$, pero $\neg(a > b) \not\Rightarrow \perp$. Contraejemplo: $a = b = 3$: $m = 3$ y $3 > 3$ es falso. Correcciones: (i) postcondición $m \geq b$ (válida en ambas ramas); (ii) precondition $a > b$: así la rama falsa se vuelve inalcanzable y la verdadera da $m = a > b$. (Con $a \neq b$ no basta: si $a < b$, $m = b$ y $b > b$ es falso.)

Problema 6.5

Demuestra $\{ \} \text{ if } x < 0 \text{ then } y := -x \text{ else } y := x \{y \geq 0 \wedge y^2 = x^2\}$ (valor absoluto).

Solución

Rama verdadera ($x < 0$): $Q[-x/y] = (-x \geq 0) \wedge (x^2 = x^2) \equiv x \leq 0$. Como $x < 0 \Rightarrow x \leq 0$: válida. **Rama falsa** ($x \geq 0$): $Q[x/y] = (x \geq 0) \wedge (x^2 = x^2) \equiv x \geq 0$: coincide con la condición de la rama. Regla condicional $\Rightarrow$ válida. Es decir, $y = |x|$.

Problema 6.6

(if sin else) Demuestra $\{ \} \text{ if } x > y \text{ then } (t := x; x := y; y := t) \{x \leq y\}$.

Solución

Regla: probar $\{P \wedge B\} C_1 \{Q\}$ y $(P \wedge \neg B) \Rightarrow Q$.

Rama verdadera, hacia atrás desde $x \leq y$: por $y := t$: $x \leq t$; por $x := y$: $y \leq t$; por $t := x$: $y \leq x$. Así $\{y \leq x\} t := x; x := y; y := t \{x \leq y\}$, y como $x > y \Rightarrow y \leq x$, por consecuencia $\{x > y\} C_1 \{x \leq y\}$.

Rama falsa (skip): $\top \wedge \neg(x > y) \equiv x \leq y \Rightarrow x \leq y$. ✓

Por la regla del if sin else, la terna es válida.

Problema 6.7

Demuestra $\{x > 0\} \text{ if } x > 10 \text{ then } x := x - 10 \text{ else } x := x + 5 \{x > 0\}$.

Solución

Rama verdadera: $P \wedge B \equiv x > 10$; $wp = x - 10 > 0 \iff x > 10$. ✓ **Rama falsa**: $P \wedge \neg B \equiv 0 < x \leq 10$; $wp = x + 5 > 0 \iff x > -5$, y $0 < x \leq 10 \Rightarrow x > -5$. ✓ Regla condicional $\Rightarrow$ válida. (Nótese que $x > 0$ es un invariante de este bloque.)

Problema 6.8 (reto)

Demuestra que el máximo de tres valores es correcto:

if $a \geq b$ **then** (**if** $a \geq c$ **then** $m := a$ **else** $m := c$) **else** (**if** $b \geq c$ **then** $m := b$ **else** $m := c$)

con $P = \top$ y $Q : m \geq a \wedge m \geq b \wedge m \geq c$.

Solución

Se aplica la regla condicional tres veces; quedan 4 hojas, cada una con su precondition acumulada:

1. $a \geq b \wedge a \geq c, m := a$: $wp = a \geq a \wedge a \geq b \wedge a \geq c$, implicada. ✓
2. $a \geq b \wedge a < c, m := c$: $wp = c \geq a \wedge c \geq b \wedge c \geq c$; $c > a \geq b$ da $c \geq a$ y $c \geq b$. ✓
3. $a < b \wedge b \geq c, m := b$: $wp = b \geq a \wedge b \geq b \wedge b \geq c$. ✓
4. $a < b \wedge b < c, m := c$: $c > b > a$. ✓

Cada if interno se demuestra con precondition $a \geq b$ (o $a < b$) y la misma Q; luego el if externo. Válida y total.

Tema 7. Ciclos: inducción, invariantes y terminación

Problema 7.1 (★ pregunta del examen del año pasado)

Demostrar que el invariante en las siguientes instrucciones es $i + j = 9$:

```
int j = 9;
for( int i=0; i<10; i++ )
    j--;
```

¿Qué valores tienen i y j al terminar? ¿Por qué termina?

Solución

Equivalente con **while**: $j:=9; i:=0; \text{ while } i<10 \text{ do } \{ j:=j-1; i:=i+1 \}$.

Sean i_n, j_n los valores tras n iteraciones. Traza: $(i, j) = (0, 9), (1, 8), (2, 7), \dots$ Sea $P(n) : i_n + j_n = 9$.

Caso base ($n = 0$, antes de entrar): $i_0 = 0, j_0 = 9 \Rightarrow i_0 + j_0 = 9$. ✓

Hipótesis de inducción: tras n iteraciones, $i_n + j_n = 9$.

Paso inductivo: si la guarda $i_n < 10$ se cumple, el cuerpo hace $j_{n+1} = j_n - 1$ e $i_{n+1} = i_n + 1$. Entonces

$$i_{n+1} + j_{n+1} = (i_n + 1) + (j_n - 1) = i_n + j_n \stackrel{\text{H.I.}}{=} 9.$$

Por inducción, $i + j = 9$ vale después de cualquier número de iteraciones: es invariante. ■

(Forma Hoare equivalente: $\{i + j = 9 \wedge i < 10\} j := j - 1; i := i + 1 \{i + j = 9\}$. Hacia atrás: por $i := i + 1$: $i + 1 + j = 9$; por $j := j - 1$: $i + 1 + j - 1 = 9 \iff i + j = 9$. ✓)

Salida: también es invariante $0 \leq i \leq 10$ (empieza en 0, sube de uno en uno y solo sube si $i < 10$).

Al salir, $\neg(i < 10) \wedge i \leq 10 \Rightarrow i = 10$, y por el invariante $j = 9 - 10 = -1$.

Terminación: variante $V = 10 - i$: entero, ≥ 0 mientras el ciclo continúa y decrece en 1 cada iteración. Se ejecutan exactamente 10 iteraciones: corrección total.

Problema 7.2 (estilo examen)

Encuentra y demuestra el invariante que relaciona k e i , y el valor final de k :

```
int k = 20;
for( int i = 0; i < 5; i++ )
    k = k - 4;
```

Solución

Traza: $(i, k) = (0, 20), (1, 16), (2, 12), (3, 8), (4, 4), (5, 0)$. Se observa $k_n = 20 - 4n$, $i_n = n$; eliminando n :
I: $k + 4i = 20$.

Base: $k_0 + 4i_0 = 20 + 0 = 20$. Paso: $k_{n+1} + 4i_{n+1} = (k_n - 4) + 4(i_n + 1) = k_n + 4i_n = 20$. Por inducción, invariante. Salida: $i = 5 \Rightarrow k = 20 - 20 = 0$. Variante $5 - i$.

Problema 7.3

Demuestra que la función calcula el cuadrado de un entero $A \geq 0$ (identifica invariante, salida y variante):

```
Function Cuadrado( A )
```

```

C ← 0; D ← 0
While ( D ≠ A )
  C ← C + A
  D ← D + 1
Return C

```

¿Qué ocurre si $A < 0$?

Solución

Traza con $A = 4$: $(C, D) = (0, 0), (4, 1), (8, 2), (12, 3), (16, 4)$: $C_n = nA$, $D_n = n \Rightarrow \mathbf{I} : C = D \cdot A$.

Base: $C_0 = 0 = 0 \cdot A = D_0 A$. **H.I.:** $C_n = D_n A$. **Paso:** $C_{n+1} = C_n + A = D_n A + A = (D_n + 1)A = D_{n+1} A$. Por inducción, I es invariante.

Salida: el ciclo termina con $D = A$; entonces $C = DA = A \cdot A = A^2$. (El invariante solo no basta: se combina con $\neg B$.)

Terminación: con $A \geq 0$, D parte de 0 y sube de 1 en 1; la variante $V = A - D$ es entera, ≥ 0 y decrece en 1: tras A iteraciones $D = A$. Corrección total.

Si $A < 0$: D nunca alcanza A (solo crece) y el ciclo **no termina**. La terna solo sería parcialmente correcta ("si terminara, $C = A^2$ ").

Problema 7.4

¿Qué realiza la subrutina? Demuéstralo (N, M enteros, $M \geq 0$).

```

Subroutine ?( N, M; R )
  R ← 1
  While ( M > 0 )
    R ← R × N
    M ← M - 1
  Return
End Subroutine

```

¿Por qué $R = N^M$ no es un invariante?

Solución

Calcula $R = N^{M_0}$, donde M_0 es el valor original de M . Traza con $N = 2$, $M_0 = 5$: $(R, M) = (1, 5), (2, 4), (4, 3), (8, 2), (16, 1), (32, 0)$; siempre $R \cdot 2^M = 32$.

Invariante: $I : R \cdot N^M = N^{M_0}$ ("calculado $\times$ pendiente = total"). **Base:** $R_0 N^{M_0} = 1 \cdot N^{M_0}$. ✓ **Paso:** si $M_n > 0$: $R_{n+1} N^{M_{n+1}} = (R_n N) N^{M_n - 1} = R_n N^{M_n} \stackrel{\text{H.I.}}{=} N^{M_0}$. ✓ **Salida:** $M = 0 \Rightarrow R \cdot N^0 = R = N^{M_0}$.

Terminación: variante $V = M$, entero ≥ 0 que decrece en 1.

$R = N^M$ no se conserva: al inicio $R = 1$ y $N^M = N^{M_0}$ (falla si $M_0 > 0$, $N \neq 1$). R crece mientras M baja: la relación conservada es el *producto* $R N^M$. Error típico: expresar el resultado con el M final (que vale 0) en lugar de M_0 .

Problema 7.5

Demuestra la corrección total de ($n \geq 0$):

```

s := 0; i := 0;
while i ≠ n do

```

```
i := i + 1;
s := s + i
```

con postcondición $s = \frac{n(n+1)}{2}$.

Solución

Invariante: $I : s = \frac{i(i+1)}{2} \wedge 0 \leq i \leq n$. **Base:** $i = 0, s = 0 = \frac{0 \cdot 1}{2}$. **✓ Paso:** si $i_n \neq n$ (y $i_n \leq n$, luego $i_n < n$): $i_{n+1} = i_n + 1 \leq n$ y $s_{n+1} = s_n + i_{n+1} = \frac{i_n(i_n+1)}{2} + (i_n + 1) = \frac{(i_n+1)(i_n+2)}{2} = \frac{i_{n+1}(i_{n+1}+1)}{2}$. **✓ Salida:** $i = n \Rightarrow s = \frac{n(n+1)}{2}$. **Variante:** $n - i$.

Problema 7.6

División entera por restas sucesivas ($a \geq 0, b > 0$):

```
q := 0; r := a;
while r ≥ b do
  r := r - b;
  q := q + 1
```

Demuestra que al terminar $q = a \text{ div } b$ y $r = a \text{ mód } b$. Traza con $a = 17, b = 5$.

Solución

Invariante: $I : a = q \cdot b + r \wedge r \geq 0$. **Base:** $q = 0, r = a: 0 \cdot b + a = a$ y $a \geq 0$. **✓ Paso:** con guarda $r \geq b$: $r' = r - b \geq 0, q' = q + 1$ y $q'b + r' = qb + b + r - b = qb + r = a$. **✓ Salida:** $I \wedge \neg(r \geq b) \Rightarrow a = qb + r \wedge 0 \leq r < b$, que es exactamente la definición de cociente y residuo. **Terminación:** variante r : entera, ≥ 0 (por I) y decrece en $b \geq 1$ cada iteración. Traza: $(q, r) = (0, 17), (1, 12), (2, 7), (3, 2)$; sale con $r = 2 < 5$: $17 = 3 \cdot 5 + 2$. **✓**

Problema 7.7

Para cada ciclo, propón el invariante y demuéstalo por inducción ($n \geq 0$): (a) $f:=1; i:=0; \text{ while } i < n \text{ do } \{ i:=i+1; f:=f \cdot i \}$ (b) $s:=0; i:=0; \text{ while } i < n \text{ do } \{ s:=s+2 \cdot i+1; i:=i+1 \}$

Solución

(a) $I : f = i!$. Base: $f = 1 = 0!$. Paso: $f_{n+1} = f_n \cdot i_{n+1} = i_n!(i_n + 1) = (i_n + 1)! = i_{n+1}!$. Salida $i = n$: $f = n!$. Variante $n - i$.
 (b) $I : s = i^2$ (suma de los primeros i impares). Base: $0 = 0^2$. Paso: $s_{n+1} = s_n + 2i_n + 1 = i_n^2 + 2i_n + 1 = (i_n + 1)^2 = i_{n+1}^2$. Salida: $s = n^2$. Variante $n - i$.

Problema 7.8

Responde: (a) ¿Por qué una tabla de valores no es una demostración? (b) ¿Qué relación hay entre el caso base/paso inductivo y las obligaciones de Hoare para **while**? (c) ¿Qué se obtiene si se prueba el invariante y la salida, pero no la variante?

Solución

(a) La tabla muestra algunos casos y sugiere el patrón; la inducción prueba que vale para *cualquier* número de iteraciones. (b) Caso base $\leftrightarrow$ inicialización ($P \Rightarrow I$); paso inductivo $\leftrightarrow$ mantenimiento $\{I \wedge B\} C \{I\}$; después se añade la salida $I \wedge \neg B \Rightarrow Q$. (c) Solo corrección *parcial*: “si el ciclo termina, el resultado es correcto”. La variante (entera, no negativa, estrictamente decreciente) aporta la terminación y con ella la corrección total.

Tema 8. Función 91 de McCarthy, inducción modificada y Ackermann**Problema 8.1**

Con $f(x) = x - 10$ si $x > 100$ y $f(x) = f(f(x + 11))$ si $x \leq 100$: (a) desarrolla a mano $f(99)$ y $f(95)$ y cuenta cuántas llamadas se hacen en cada caso; (b) desarrolla $f(89)$ mostrando solo los pasos intermedios importantes; (c) calcula $f(105)$ y $f(101)$.

Solución

(a) $f(99) = f(f(110)) = f(100) = f(f(111)) = f(101) = 91$. Llamadas: $f(99), f(110), f(100), f(111), f(101)$: **5**.
 $f(95) = f(f(106)) = f(96) = f(f(107)) = f(97) = \dots = f(100) = f(f(111)) = f(101) = 91$. Se llama a $f(95), \dots, f(101)$ (7) y a $f(106), \dots, f(111)$ (6): **13** llamadas.
 (b) $f(89) = f(f(100))$; como $f(100) = 91$ (3 llamadas), queda $f(91) = f(f(102)) = f(92) = \dots = f(100) = f(101) = 91$. (Total: 25 llamadas.)
 (c) $f(105) = 95$; $f(101) = 91$. En general $f(x) = x - 10$ para $x > 100$ y $f(x) = 91$ para $x \leq 101$.

Problema 8.2

Enuncia el principio de inducción matemática modificado y úsalo para demostrar que $f(x) = 91$ para todo entero $x \leq 100$. Explica por qué la inducción simple (“si vale en n vale en $n + 1$ ”) no basta.

Solución

Principio modificado. Sea k un entero fijo. Si (1) $P(k)$ es verdadera y (2) para todo $n < k$, $[P(m)$ verdadera para todo m con $n < m \leq k]$ implica $P(n)$, entonces $P(n)$ vale para todo $n \leq k$.

Demostración con $k = 100$ y $P(x) : f(x) = 91$.

Paso básico: $f(100) = f(f(111)) = f(101) = 91$ (pues $111 > 100$ y $101 > 100$).

Hipótesis: sea $x < 100$ y supongamos $f(m) = 91$ para todo m con $x < m \leq 100$.

Paso inductivo: como $x \leq 100$, $f(x) = f(f(x + 11))$.

- **Caso a)** $x + 11 > 100$ (es decir, $90 \leq x \leq 99$): $f(x + 11) = x + 1$, luego $f(x) = f(x + 1)$. Como $x < x + 1 \leq 100$, por H.I. $f(x + 1) = 91$.
- **Caso b)** $x + 11 \leq 100$ ($x \leq 89$): como $x < x + 11 \leq 100$, por H.I. $f(x + 11) = 91$, luego $f(x) = f(91)$. Como $x \leq 89 < 91 \leq 100$, por H.I. $f(91) = 91$.

En ambos casos $f(x) = 91$. Por el principio modificado, $f(x) = 91$ para todo $x \leq 100$. ■

Por qué no basta la inducción simple: el caso b) necesita el valor de f en *dos* puntos mayores que x ($x + 11$ y 91), no solo en $x + 1$; la hipótesis debe cubrir todo el tramo $(x, 100]$. Además la recursión

“se aleja” (el argumento sube) antes de resolverse, así que no hay un contador que baje de uno en uno.

Problema 8.3 (estilo examen)

Sea $g(x) = x - 2$ si $x > 20$ y $g(x) = g(g(x + 3))$ si $x \leq 20$. Calcula $g(20)$, $g(19)$, $g(15)$; conjetura el valor de $g(x)$ para $x \leq 20$ y demuéstralo con inducción modificada.

Solución

$g(20) = g(g(23)) = g(21) = 19$; $g(19) = g(g(22)) = g(20) = 19$; $g(15) = g(g(18)) = g(19) = 19$ (usando $g(18) = g(g(21)) = g(19) = 19$). **Conjetura:** $g(x) = 19$ para todo $x \leq 20$.

Paso básico ($k = 20$): $g(20) = 19$ (calculado). **H.I.:** para $x < 20$, $g(m) = 19$ para todo m con $x < m \leq 20$. **Paso:** $g(x) = g(g(x + 3))$.

- Caso a) $x + 3 > 20$ ($x \in \{18, 19\}$): $g(x + 3) = x + 1$, así $g(x) = g(x + 1)$ con $x < x + 1 \leq 20$: por H.I. vale 19.
- Caso b) $x + 3 \leq 20$ ($x \leq 17$): $x < x + 3 \leq 20 \Rightarrow g(x + 3) = 19$; luego $g(x) = g(19)$ y $x \leq 17 < 19 \leq 20 \Rightarrow g(19) = 19$.

Por inducción modificada, $g(x) = 19$ para todo $x \leq 20$. ■ (Patrón general del tipo McCarthy: techo c , resta a , suma $b > a$: valor $c + b - 2a$; aquí $20 + 3 - 4 = 19$ y $100 + 11 - 20 = 91$.)

Problema 8.4

Con la definición $A(0, y) = y + 1$, $A(x + 1, 0) = A(x, 1)$, $A(x + 1, y + 1) = A(x, A(x + 1, y))$: (a) calcula $A(1, 2)$ y $A(2, 1)$ aplicando *solo* la definición; (b) demuestra por inducción sobre z que $A(1, z) = z + 2$ y $A(2, z) = 2z + 3$.

Solución

(a) $A(1, 0) = A(0, 1) = 2$; $A(1, 1) = A(0, A(1, 0)) = A(0, 2) = 3$; $A(1, 2) = A(0, A(1, 1)) = A(0, 3) = 4$.

$A(2, 0) = A(1, 1) = 3$; $A(2, 1) = A(1, A(2, 0)) = A(1, 3) = A(0, A(1, 2)) = A(0, 4) = 5$.

(b) $A(1, z) = z + 2$. Base: $A(1, 0) = A(0, 1) = 2$. H.I.: $A(1, n) = n + 2$. Paso: $A(1, n + 1) = A(0, A(1, n)) = A(0, n + 2) = n + 3 = (n + 1) + 2$. ✓

$A(2, z) = 2z + 3$. Base: $A(2, 0) = A(1, 1) = 3$. H.I.: $A(2, n) = 2n + 3$. Paso: $A(2, n + 1) = A(1, A(2, n)) = A(1, 2n + 3) = (2n + 3) + 2 = 2(n + 1) + 3$, usando el resultado anterior como lema. ✓

Problema 8.5

Demuestra que $A(3, z) = 2^{z+3} - 3$. Calcula $A(3, 3)$ y $A(4, 1)$. ¿Por qué el programa recursivo de Ackermann termina en `StackOverflowException` para valores pequeños como $(4, 2)$?

Solución

Base: $A(3, 0) = A(2, 1) = 5 = 2^3 - 3$. Paso: $A(3, n + 1) = A(2, A(3, n)) = 2(2^{n+3} - 3) + 3 = 2^{n+4} - 6 + 3 = 2^{n+4} - 3$. ✓

$A(3, 3) = 2^6 - 3 = 61$. $A(4, 0) = A(3, 1) = 13$; $A(4, 1) = A(3, A(4, 0)) = A(3, 13) = 2^{16} - 3 = 65533$. En general $A(4, z)$ es una torre de $z + 3$ doses menos 3; $A(4, 2) = 2^{65536} - 3$ tiene 19 729 dígitos.

La recursión es doble y el argumento interno crece de forma no primitiva recursiva: la profundi-

dad de la pila y el número de llamadas son astronómicos antes de llegar a un caso base; ninguna pila finita alcanza.

Problema 8.6

Usando los lemas del problema anterior, demuestra $\{x = 2\} w := A(x, y) + 1 \{w = 2y + 4\}$ y $\{x = 1\} w := A(x, y) \{w = y + 2\}$. ¿Por qué no basta el axioma de asignación?

Solución

Primera: por asignación, $(w = 2y + 4)[A(x, y) + 1/w] \equiv A(x, y) + 1 = 2y + 4 \equiv A(x, y) = 2y + 3$, así $\{A(x, y) = 2y + 3\} w := A(x, y) + 1 \{w = 2y + 4\}$. Por el lema $A(2, z) = 2z + 3$: $x = 2 \Rightarrow A(x, y) = 2y + 3$. Por consecuencia:

$$\frac{x = 2 \Rightarrow A(x, y) = 2y + 3 \quad \{A(x, y) = 2y + 3\} w := A(x, y) + 1 \{w = 2y + 4\}}{\{x = 2\} w := A(x, y) + 1 \{w = 2y + 4\}}.$$

Segunda: idéntica con $\{A(x, y) = y + 2\} w := A(x, y) \{w = y + 2\}$ y el lema $x = 1 \Rightarrow A(x, y) = y + 2$. El axioma solo produce la precondition $A(x, y) = \dots$; para llegar a $x = 1$ o $x = 2$ hace falta un *teorema* sobre A (demostrado por inducción aparte) y la regla de consecuencia. Así se reutilizan lemas sin volver a desenrollar la recursión.

Problema 8.7

Compara la inducción usada para ciclos con la usada para f (McCarthy) y Ackermann: dirección, hipótesis y qué se demuestra.

Solución

	Ciclos	McCarthy / Ackermann
Dirección	sube desde $n = 0$ (iteraciones)	McCarthy: baja desde un techo k ; Ackermann: inducción anidada sobre x y, dentro, sobre y
Hipótesis	$P(n)$ para el n anterior	$P(m)$ para <i>todo</i> $m \in (x, k]$ (fuerte)
Qué prueba	un invariante se conserva	la función devuelve cierto valor/fórmula
Ejemplo	$C = DA$ en Cuadrado	$f(x) = 91$; $A(2, z) = 2z + 3$

Tema 9. Programación lógica y Prolog

Problema 9.1

(a) Escribe como cláusula (disyunción de literales) y como implicación la regla $\text{abuelo}(X, Z) :- \text{padre}(X, Y), \text{padre}(Y, Z)$. (b) ¿Qué es una cláusula de Horn definida? ¿Y una cláusula objetivo? Representa la consulta $?- \text{abuelo}(\text{juan}, W)$.

Solución

- (a) Implicación: $\forall X, Y, Z \ (padre(X, Y) \wedge padre(Y, Z) \rightarrow abuelo(X, Z))$. Cláusula: $abuelo(X, Z) \vee \neg padre(X, Y) \vee \neg padre(Y, Z)$.
- (b) **Horn**: a lo más un literal positivo. **Definida**: exactamente uno (hechos y reglas; la cabeza es el positivo). **Objetivo**: ningún literal positivo; la consulta es $\leftarrow abuelo(juan, W) \equiv \neg abuelo(juan, W)$. Un hecho $padre(juan, pedro)$ es $padre(juan, pedro) \leftarrow \top$.

Problema 9.2

Da el unificador más general o explica por qué no existe: (a) $p(X, b)$ y $p(a, Y)$; (b) $f(X, g(Y))$ y $f(h(Z), g(a))$; (c) $p(X, X)$ y $p(a, b)$; (d) $gusta(maria, X)$ y $gusta(Y, cine)$; (e) $q(f(X), X)$ y $q(Y, a)$; (f) $p(X)$ y $p(f(X))$.

Solución

- (a) $\{X/a, Y/b\}$. (b) $\{X/h(Z), Y/a\}$. (c) No: X/a obliga a $a = b$, constantes distintas. (d) $\{Y/maria, X/cine\}$. (e) $\{Y/f(a), X/a\}$ (tras X/a , $f(X)$ se vuelve $f(a)$). (f) No (prueba de ocurrencia): $X = f(X)$ exigiría un término infinito.

Problema 9.3

Demuestra por **refutación** con resolución: (a) $S = \{\neg hombre(X) \vee mortal(X), hombre(socrates)\} \models mortal(socrates)$. (b) $S = \{P(X) \vee Q(X), \neg Q(X) \vee R(X), \neg R(X) \vee P(X)\} \models P(X)$.

Solución

Se agrega la negación de la conclusión y se busca la cláusula vacía $\square$.

- (a) $C_1 : \neg hombre(X) \vee mortal(X)$, $C_2 : hombre(socrates)$, $C_3 : \neg mortal(socrates)$. C_3, C_1 con $\theta = \{X/socrates\}$: $\neg hombre(socrates)$; con C_2 : $\square$. Luego $S \models mortal(socrates)$.
- (b) $C_4 : \neg P(X)$. $C_4, C_3 \Rightarrow \neg R(X)$; $\neg R(X), C_2 \Rightarrow \neg Q(X)$; $\neg Q(X), C_1 \Rightarrow P(X)$; $P(X), C_4 \Rightarrow \square$. $S \cup \{\neg P(X)\}$ es insatisfacible $\Rightarrow S \models P(X)$.

Problema 9.4

Dado el programa

```
padre(juan, pedro).
padre(pedro, luis).
padre(pedro, ana).
abuelo(X, Z) :- padre(X, Y), padre(Y, Z).
```

- (a) Escribe la derivación SLD para $?- abuelo(juan, W)$. indicando en cada paso la cláusula usada y el unificador. (b) ¿Qué responde Prolog al teclear ;?

Solución

Renombramos la regla: $abuelo(X_1, Z_1) \leftarrow padre(X_1, Y_1), padre(Y_1, Z_1)$.

- $G_0 : \leftarrow abuelo(juan, W)$. Con la regla, $\theta_1 = \{X_1/juan, Z_1/W\}$.
- $G_1 : \leftarrow padre(juan, Y_1), padre(Y_1, W)$. Primera meta con el hecho $padre(juan, pedro)$: $\theta_2 = \{Y_1/pedro\}$.

3. $G_2 : \leftarrow \text{padre}(\text{pedro}, W)$. Con $\text{padre}(\text{pedro}, \text{luis})$: $\theta_3 = \{W/\text{luis}\}$.

4. $G_3 : \square$. Respuesta: $W = \text{luis}$.

(b) Con ; Prolog retrocede al último punto de elección (G_2) y usa $\text{padre}(\text{pedro}, \text{ana})$: $W = \text{ana}$. Con otro ; no quedan cláusulas para $\text{padre}(\text{pedro}, W)$ ni para $\text{padre}(\text{juan}, Y_1)$: false.

Problema 9.5

Explica por qué el paso “reemplazar una meta por el cuerpo de una regla” es una instancia de la regla de resolución. ¿Qué significan las letras S, L, D?

Solución

La regla $A \leftarrow B_1, \dots, B_m$ es la cláusula $A \vee \neg B_1 \vee \dots \vee \neg B_m$ y la meta $\leftarrow A_1, \dots, A_k, \dots, A_n$ es $\neg A_1 \vee \dots \vee \neg A_k \vee \dots \vee \neg A_n$. Si $\theta = \text{UMG}(A, A_k)$, los literales A y $\neg A_k$ son complementarios: se eliminan y el resolvente es $(\neg A_1 \vee \dots \vee \neg A_{k-1} \vee \neg B_1 \vee \dots \vee \neg B_m \vee \neg A_{k+1} \vee \dots \vee \neg A_n)\theta$, que en notación de metas es $\leftarrow (A_1, \dots, A_{k-1}, B_1, \dots, B_m, A_{k+1}, \dots, A_n)\theta$.

S: regla de *selección* de la meta (Prolog: la de más a la izquierda). **L:** *lineal* (cada resolvente se obtiene del anterior y una cláusula del programa). **D:** cláusulas *definidas*. Prolog además prueba las cláusulas en orden de programa, en profundidad, con retroceso.

Problema 9.6

Dado el programa, indica si cada consulta tiene éxito, falla o produce error, y justifica:

```
a.
b.
c :- fail.
s :- a, \+ c.
t :- c ; b.
u :- a, c.
v :- \+ a ; c.
w :- b, ( \+ c, a ).
x :- y.
```

Consultas: ?- s. ?- t. ?- u. ?- v. ?- w. ?- x.

Solución

s ✓ a es hecho; c no se puede demostrar (su cuerpo es fail), así que \+ c tiene éxito.
t ✓ disyunción: c falla, se prueba la alternativa b, que es hecho.
u × a tiene éxito pero c falla.
v × \+ a falla (porque a sí se demuestra) y c también falla.
w ✓ b; luego \+ c tiene éxito y a es hecho.
x error y no está definido: SWI-Prolog lanza *Unknown procedure: y/0* (existence_error); no es lo mismo que “falso”.

Recordatorio: \+ G es *negación como falla*: tiene éxito si el programa *no logra demostrar G*.

Problema 9.7

Con el programa `MariaGusta.pl`

```
gusta( maria, helado ).
gusta( maria, leer ).
gusta( maria, cine ).
dar( juan, beso, maria ).
recibe( X, beso ) :- dar( juan, beso, X ).
```

indica la respuesta de: (a) `gusta(maria, cine)`. (b) `gusta(juan, cine)`. (c) `gusta(Quien, helado)`. (d) `gusta(maria, Maria)`. (con `;`) (e) `recibe(X, beso)`. (f) `recibe ( X, beso )`.

Solución

(a) true (átomos que coinciden). (b) false. (c) `Quien = maria`. (d) `Maria` es una *variable* (mayúscula inicial), no el átomo `maria`: responde `Maria = helado ; Maria = leer ; Maria = cine`. (e) Se resuelve con la regla: la meta `dar( juan, beso, X )` unifica con el hecho: `X = maria`. (f) **Error de sintaxis** (*operator expected*): no puede haber espacio entre el nombre del predicado y el paréntesis.

Problema 9.8 (estilo examen)

Considera

```
maximo( X, Y, X ) :- X >= Y.
maximo( X, Y, Y ) :- X < Y.
mayor( X, Y, X ) :- X >= Y, !.
mayor( X, Y, Y ) :- X < Y.
max( X, Y, Y ) :- X < Y, !.
max( X, _, X ).
```

(a) ¿Qué hace el corte `!`? (b) ¿Qué es un corte “verde” y uno “rojo”? Clasifica los de `mayor` y `max`. (c) ¿Qué responde `?- max(3, 5, 3)`? ¿Es correcto? Corrígelo.

Solución

(a) El corte tiene éxito una vez y *elimina los puntos de elección* creados desde que se eligió la cláusula actual: Prolog ya no probará otras cláusulas del mismo predicado ni otras alternativas de las metas anteriores al `!` dentro del cuerpo.

(b) **Verde**: solo mejora eficiencia; quitarlo no cambia las respuestas. **Rojo**: cambia el significado; quitarlo cambia las respuestas. En `mayor` es verde (las condiciones $X \geq Y$ y $X < Y$ son excluyentes; el corte evita probar una cláusula que igual fallaría). En `max` es rojo: la segunda cláusula no tiene condición y depende del corte (sin él, `max(3, 5, M)` daría $M = 5 ; M = 3$).

(c) `?- max(3, 5, 3)`. La primera cláusula exige unificar el tercer argumento con $Y = 5$: $3 \neq 5$, falla *antes* de llegar al corte. La segunda, `max(X, _, X)`, unifica con $X = 3$: responde true. **Incorrecto** (el máximo es 5). Corrección: hacer la unificación de la salida *después* del corte:

```
max( X, Y, Z ) :- X < Y, !, Z = Y.
max( X, _, X ).
```

o usar la versión `maximo` sin cortes.

Problema 9.9

Indica la respuesta de SWI-Prolog: (a) $X = 2+3$. (b) $X \text{ is } 2+3$. (c) $5 \text{ is } 2+3$. (d) $2+3 = 5$. (e) $2+3 ::= 5$. (f) $X \text{ is } Y+1$.

Solución

(a) $X = 2+3$ (unifica con el *término* sin evaluar). (b) $X = 5$ (is evalúa el lado derecho). (c) true. (d) false (el término $+(2,3)$ no es el número 5). (e) true ($::=$ compara valores aritméticos). (f) Error: *arguments are not sufficiently instantiated* (Y no tiene valor).

Problema 9.10

(a) Traza `?- factorial(3, F).` con

```
factorial(0, 1) :- !.
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.
```

(b) Escribe `suma_hasta(N, S)` ($S = 1 + 2 + \dots + N$) y `mcd(A, B, G)` (algoritmo de Euclides) con el mismo estilo.

Solución

(a) `factorial(3,F)`: $3 > 0$, $N1 = 2$, llama `factorial(2,F1) → N1 = 1`, `factorial(1,F1') → N1 = 0`, `factorial(0,F1'')`: primera cláusula, $F1'' = 1$ y el corte descarta la segunda. Al regresar: $F1' = 1 \cdot 1 = 1$, $F1 = 2 \cdot 1 = 2$, $F = 3 \cdot 2 = 6$.

```
suma_hasta(0, 0) :- !.
suma_hasta(N, S) :- N > 0, N1 is N - 1, suma_hasta(N1, S1), S is S1 + N.

mcd(A, 0, A) :- !.
mcd(A, B, G) :- B > 0, R is A mod B, mcd(B, R, G).
```

`?- mcd(48, 18, G).` $\rightarrow \text{mcd}(18, 12) \rightarrow \text{mcd}(12, 6) \rightarrow \text{mcd}(6, 0) \rightarrow G = 6$. Mismo patrón que en clase: caso base protegido por corte y caso recursivo que reduce el problema.

Problema 9.11

Compara resolución general y resolución SLD (qué cláusulas usan, forma de la derivación, qué significa el éxito, cómo manejan alternativas). ¿Por qué el orden de las cláusulas importa en Prolog aunque lógicamente no importe?

Solución

	General	SLD / Prolog
Cláusulas	arbitrarias	definidas + una meta
Elección	cualquier par de cláusulas	la meta seleccionada con una cláusula del programa
Derivación	árbol/grafó amplio	lineal por rama
Éxito	se deriva $\square$	la lista de metas queda vacía
Alternativas	muchas combinaciones	retroceso (backtracking)
Lógicamente el conjunto de consecuencias no cambia al reordenar. Operacionalmente Prolog elige		

la meta de más a la izquierda y prueba cláusulas en orden, en profundidad: un orden desafortunado puede ser lento o *no terminar* (p. ej. una regla recursiva por la izquierda como $\text{anc}(X, Y) :- \text{anc}(X, Z), \text{padre}(Z, Y)$. antes del caso base). Además, que una búsqueda no termine no significa que la consulta sea falsa.

Tema 10. Exámenes simulados

Resuelve cada simulacro en **90 minutos**, sin notas, escribiendo las demostraciones completas como si fuera el examen real. Los problemas se numeran como **10.1–10.6** (simulacro A) y **10.7–10.12** (simulacro B).

Simulacro A

Problema 10.1 · A1

Demostrar que la siguiente terna de Hoare es válida:

$$\{a \neq b\} \text{ if } (a > b) \text{ then } (m := a; n := b) \text{ else } (m := b; n := a) \{m > n\}.$$

Solución

$P : a \neq b, B : a > b, Q : m > n$.

Rama verdadera $C_1 : m := a; n := b$. Hacia atrás: por $n := b: m > b$; por $m := a: a > b$. Así $\{a > b\} C_1 \{m > n\}$, y $P \wedge B \equiv a \neq b \wedge a > b \Rightarrow a > b$. ✓

Rama falsa $C_2 : m := b; n := a$. Hacia atrás: por $n := a: m > a$; por $m := b: b > a$. Así $\{b > a\} C_2 \{m > n\}$. Además $P \wedge \neg B \equiv a \neq b \wedge a \leq b \equiv a < b \Rightarrow b > a$. ✓

Por la regla condicional (y composición en cada rama), la terna es válida; sin ciclos, corrección total. Nótese que la precondition $a \neq b$ es *necesaria*: con $a = b$ se obtendría $m = n$.

Problema 10.2 · A2

¿Qué es la corrección parcial y la corrección total de un programa? Da un ejemplo de programa parcialmente correcto que no sea totalmente correcto.

Solución

Parcial: $\{P\} C \{Q\}$ es válida: si C inicia en P y *termina*, termina en Q . Total: además C termina para toda entrada en P . Total = parcial + terminación.

Ejemplo: $\{\top\} \text{ while } x \neq 0 \text{ do } x := x - 2 \{x = 0\}$. Es parcialmente correcto: si el ciclo termina, es porque $x = 0$ (invariante trivial $\top$, salida $\neg(x \neq 0) \Rightarrow x = 0$). No es totalmente correcto: con $x = 3$ o $x = -2$ nunca termina. Con precondition $x \geq 0 \wedge x$ par, la variante x da terminación. Otro ejemplo: Cuadrado(A) con $A < 0$.

Problema 10.3 · A3

Escribe en pseudocódigo un algoritmo de divide y vencerás que calcule la suma de los elementos de $A[l..r]$. Plantea su recurrencia y traza la llamada sobre $A = [4, 7, 1, 3]$.

Solución

```

Funcion Suma( A, l, r ): entero
Inicio
  Si l = r entonces devolver( A[l] )           // problema simple
  m = (l + r) div 2                             // descomponer en k = 2
  S1 = Suma( A, l, m )

```

```

    S2 = Suma( A, m+1, r )
    devolver( S1 + S2 )           // combinar
Fin

```

$T(1) = \Theta(1)$, $T(n) = 2T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(n)$. Traza: $\text{Suma}(1,4) \rightarrow \text{Suma}(1,2) = \text{Suma}(1,1) + \text{Suma}(2,2) = 4 + 7 = 11$; $\text{Suma}(3,4) = 1 + 3 = 4$; resultado $11 + 4 = 15$.

Problema 10.4 · A4

Demostrar que el invariante de las siguientes instrucciones es $s = 2i$ y deducir el valor final de s ($n \geq 0$):

```

int s = 0;
for( int i = 0; i < n; i++ )
    s = s + 2;

```

Solución

Equivalente: $s := 0$; $i := 0$; while $i < n$ do { $s := s + 2$; $i := i + 1$ }. **Base:** $s_0 = 0 = 2 \cdot 0 = 2i_0$. **H.I.:** $s_n = 2i_n$. **Paso:** $s_{n+1} = s_n + 2 = 2i_n + 2 = 2(i_n + 1) = 2i_{n+1}$. Por inducción, $s = 2i$ es invariante. Además $0 \leq i \leq n$ es invariante; al salir $i = n$ y por tanto $s = 2n$. Variante: $n - i$.

Problema 10.5 · A5

Con el programa

```

progenitor(ana, bea).
progenitor(ana, carlos).
progenitor(carlos, diana).
progenitor(bea, emilio).
abuela(X, Z) :- progenitor(X, Y), progenitor(Y, Z).

```

- (a) Escribe la derivación SLD de $?- \text{abuela}(\text{ana}, N)$. y todas las respuestas que da Prolog con ;.
 (b) ¿Qué responde $?- \text{abuela}(X, \text{diana})$?

Solución

(a) Regla renombrada: $\text{abuela}(X_1, Z_1) \leftarrow \text{progenitor}(X_1, Y_1), \text{progenitor}(Y_1, Z_1)$. $G_0 : \leftarrow \text{abuela}(\text{ana}, N)$; $\theta_1 = \{X_1/\text{ana}, Z_1/N\}$.
 $G_1 : \leftarrow \text{progenitor}(\text{ana}, Y_1), \text{progenitor}(Y_1, N)$. Primer hecho que unifica: $\text{progenitor}(\text{ana}, \text{bea})$, $\theta_2 = \{Y_1/\text{bea}\}$.
 $G_2 : \leftarrow \text{progenitor}(\text{bea}, N)$; con $\text{progenitor}(\text{bea}, \text{emilio})$: $\theta_3 = \{N/\text{emilio}\}$; $G_3 = \square$. $N = \text{emilio}$.
 ;: no hay otro $\text{progenitor}(\text{bea}, _)$; se retrocede a G_1 y se usa $\text{progenitor}(\text{ana}, \text{carlos})$: Y_1/carlos , $G'_2 : \leftarrow \text{progenitor}(\text{carlos}, N)$, N/diana : $N = \text{diana}$. Otro ;: false.
 (b) $G_1 : \leftarrow \text{progenitor}(X, Y_1), \text{progenitor}(Y_1, \text{diana})$. Se prueban los hechos en orden: $(\text{ana}, \text{bea}) \rightarrow \text{progenitor}(\text{bea}, \text{diana})$ falla; $(\text{ana}, \text{carlos}) \rightarrow \text{progenitor}(\text{carlos}, \text{diana})$ ✓: $X = \text{ana}$. Con ; las demás ramas ($Y_1 = \text{diana}$, $Y_1 = \text{emilio}$) fallan: false.

Problema 10.6 · A6

En el problema de las ocho reinas, explica para qué sirven $j - k$ y $j + k$, por qué basta el vector $sol[k]$ y qué parte del algoritmo realiza el “backtracking”.

Solución

Se coloca una reina por fila k (así nunca comparten fila) y $sol[k] = j$ guarda su columna: esto describe completamente el tablero. Dos reinas se atacan si comparten columna (j), la diagonal donde $j - k$ es constante o la diagonal donde $j + k$ es constante; guardando esos tres identificadores ocupados, verificar una casilla no requiere recorrer el tablero (poda inmediata). El *backtracking* es: probar columnas en orden, anotar la elegida (*push_back*), bajar a la fila $k + 1$ y, al regresar (por éxito o porque no hubo columna segura), deshacer (*pop_back*) y probar la siguiente columna.

Simulacro B

Problema 10.7 · B1

Encuentra la precondition más débil P tal que $\{P\} x := 2x + 1; y := x - 3 \{y > 4\}$ sea válida. ¿Es válida $\{x = 5\} x := 2x + 1; y := x - 3 \{y > 4\}$?

Solución

Por $y := x - 3: x - 3 > 4 \iff x > 7$. Por $x := 2x + 1: 2x + 1 > 7 \iff x > 3$. $P: x > 3$.

Composición: $\{x > 3\} x := 2x + 1 \{x > 7\}$ y $\{x > 7\} y := x - 3 \{y > 4\}$. Como $x = 5 \Rightarrow x > 3$, por fortalecimiento de la precondition la segunda terna es **válida** (de hecho $x = 11, y = 8$).

Problema 10.8 · B2

Describe el algoritmo de backtracking para encontrar *todas* las soluciones de un problema. Aplícalo al problema de las 4 reinas: ¿cuáles son las soluciones?

Solución

```

Funcion Backtracking( Etapa i, solucionesCompletas ) devuelve: boolean
Inicio
  Éxito = falso;
  IniciarOpciones( i, GrupoOpciones o );
  Repetir
    SeleccionarNuevaOpcion( o, Opcion n );
    Si ( Aceptable( n ) ) entonces
      AnotarOpcion( i, n );
      Si SolucionCompleta( i ) entonces
        Éxito = verdadero;
        ApuntarSoluciónCompleta( i, solucionesCompletas );
      Sino
        Éxito = Backtracking( i+1 );
        Si Éxito = false entonces cancelamosAnotacion( i, n ); finsi;
      Finsi;
    Finsi;
  Hasta ( NoQuedanOpciones( o ) );
  Retorna Éxito;
Fin

```

A diferencia de “una solución”, el ciclo no se detiene con el primer éxito: termina solo cuando no quedan opciones, y cada solución completa se agrega a la colección.

4 reinas (etapa = fila, opción = columna, aceptable = columna y diagonales libres): soluciones $sol = [2, 4, 1, 3]$ y $[3, 1, 4, 2]$, es decir, casillas $(0, 2), (1, 4), (2, 1), (3, 3)$ y $(0, 3), (1, 1), (2, 4), (3, 2)$.

Problema 10.9 · B3

Suponiendo demostrado que $A(1, z) = z + 2$, demuestra que $A(2, z) = 2z + 3$ para todo $z \geq 0$, calcula $A(2, 5)$ y demuestra $\{x = 1\} w := A(x, y) \{w = y + 2\}$.

Solución

Base: $A(2, 0) = A(1, 1) = 3 = 2 \cdot 0 + 3$. H.I.: $A(2, n) = 2n + 3$. Paso: $A(2, n+1) = A(1, A(2, n)) = A(1, 2n+3) = 2n+5 = 2(n+1) + 3$. Por inducción, $A(2, z) = 2z + 3$. $A(2, 5) = 13$.

Terna: por asignación, $\{A(x, y) = y + 2\} w := A(x, y) \{w = y + 2\}$. Como $x = 1 \Rightarrow A(x, y) = A(1, y) = y + 2$, por la regla de consecuencia $\{x = 1\} w := A(x, y) \{w = y + 2\}$. ■

Problema 10.10 · B4

Para el ciclo de división entera por restas ($a \geq 0, b > 0$)

```
q := 0; r := a;
while r ≥ b do { r := r - b; q := q + 1 }
```

encuentra el invariante, demuéstralo por inducción y prueba la corrección total respecto de Q : $a = qb + r \wedge 0 \leq r < b$.

Solución

Ver problema 7.6. Resumen: $I : a = qb + r \wedge r \geq 0$. Base: $a = 0 \cdot b + a, a \geq 0$. Paso: con $r_n \geq b, r_{n+1} = r_n - b \geq 0$ y $q_{n+1}b + r_{n+1} = q_nb + b + r_n - b = a$. Salida: $I \wedge r < b \Rightarrow Q$. Terminación: variante $r \geq 0$ que decrece en $b \geq 1$.

Problema 10.11 · B5

(a) ¿Qué responde Prolog a $?- \max(3, 5, 3)$. con $\max(X, Y, Y) :- X < Y, !.$ y $\max(X, _, X) .?$ Explica y corrige. (b) ¿Qué diferencia hay entre $X = 3*2, X \text{ is } 3*2$ y $3*2 ::= 6$?

Solución

(a) Responde true (incorrecto): la primera cláusula falla al unificar el tercer argumento (3 con $Y = 5$) antes de llegar al corte, y la segunda acepta cualquier caso. Es un *corte rojo* mal usado. Corrección: $\max(X, Y, Z) :- X < Y, !, Z = Y. \max(X, _, X)$. (ver problema 9.8).

(b) $X = 3*2$ liga X al término $3*2$ sin evaluar; $X \text{ is } 3*2$ evalúa y liga $X = 6$; $3*2 ::= 6$ compara valores aritméticos: true.

Problema 10.12 · B6

Demuestra con el principio de inducción modificado que la función 91 de McCarthy cumple $f(x) = 91$ para todo entero $x \leq 100$.

Solución

Ver la demostración completa del problema 8.2 (paso básico $f(100) = 91$; hipótesis sobre todo $m \in (x, 100]$; casos $x + 11 > 100$ y $x + 11 \leq 100$). En el examen escribe los tres elementos explícitamente: *principio usado, paso básico, hipótesis, paso inductivo por casos y conclusión*, verificando en cada uso de la hipótesis que el argumento cae en $(x, 100]$.

Lista de verificación antes del examen

- Sé escribir de memoria: esquema de divide y vencerás; backtracking para una, todas y la mejor solución.
- Sé enunciar: axioma de asignación, consecuencia, composición, **if** con y sin **else**, obligaciones del **while**, inducción ordinaria y modificada.
- Reconozco una rama inalcanzable ($P \wedge B \equiv \perp$) y sé justificarla.
- Sé encontrar un invariante trazando 3–4 iteraciones y eliminando n ; siempre incluyo salida y variante.
- Sé derivar $A(1, z)$, $A(2, z)$, $A(3, z)$ y la prueba de $f(x) = 91$.
- Sé pasar reglas de Prolog a cláusulas, unificar, hacer una derivación SLD y predecir la consola ($;$, $!$, $\backslash +$, $i s$, mayúsculas).
- Sé definir cohesión, acoplamiento y métricas de eficiencia (tiempo, memoria).